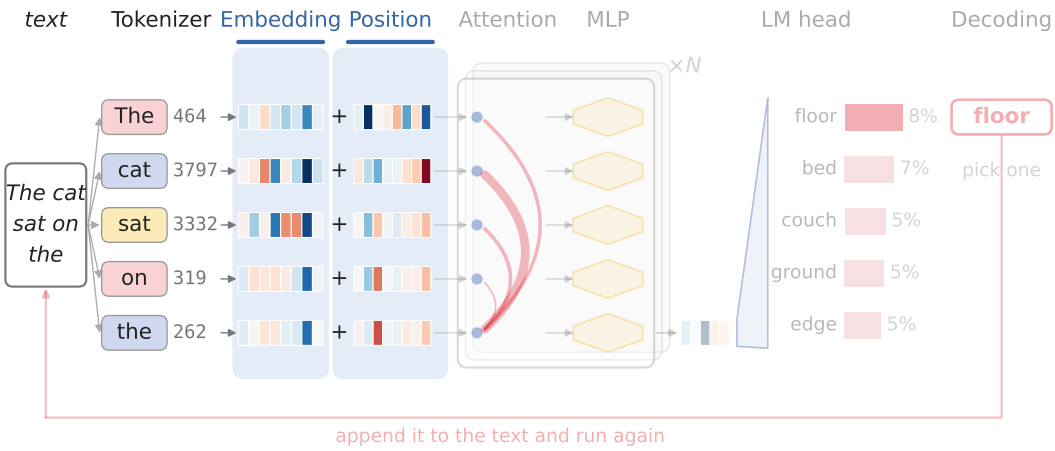
Embeddings and position

From IDs to vectors

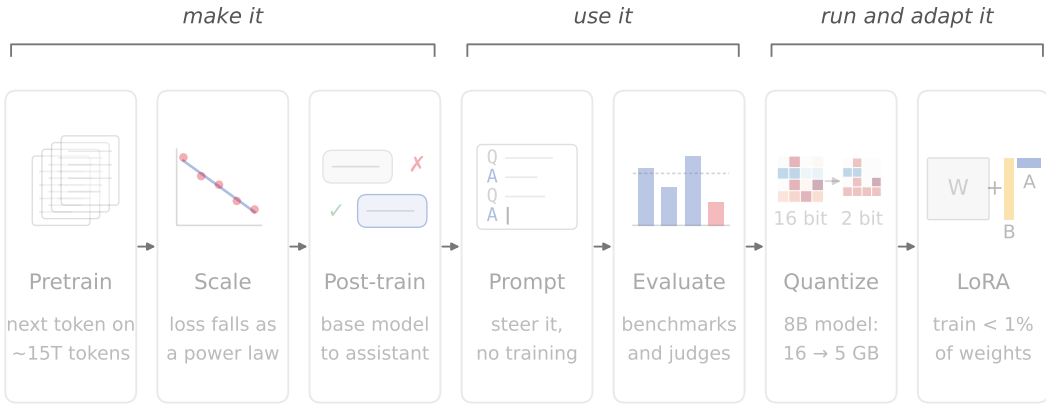
LLM chapter, session 2. Every GPT-2 number on these slides is read from its real weights.

Where we are: one forward pass



Today: the next two boxes - an ID becomes a vector, and the vector learns where it sits.

Where we are: the life of a model



Later: where the weights come from, and what you do with them once you have them.

Same words, opposite news

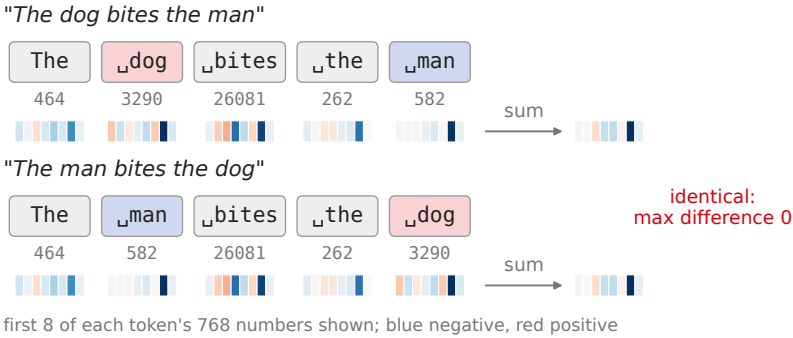
"The dog bites the man"
"The man bites the dog"

Predict: the model's first step looks every token up in a table and gets back a list of numbers. Are the two sentences still different after that step?

ուրա պարավը, բերեք ատամը քաշեմ

karginhaghordum.am/sketch/3bEwVKXicsQ

Same five numbers, same five vectors



first 8 of each token's 768 numbers shown; blue negative, red positive

The same five token IDs in a different order. Each ID fetches its row of the table, so both sentences arrive as the **same five vectors**. Add them up and the totals agree to the last bit.

Aside: without "The", "dog bites man" and "man bites dog" are not even the same tokens. The first word has no space before it, and "dog" is a different token from "␣dog" (LLM-1).

Outline

An ID becomes a vector

Where is the order?

Option 1: learn the positions

Option 2: write the positions down

Option 3: rotate, do not add

Longer and longer

An ID becomes a vector

3290 is a row number, not a quantity.

An ID is a name, not a number

- LLM-1 ended with integers: " cat" is **3797**, " dog" is **3290**.
- Is a cat 507 "more" than a dog? Is " dog" + 1 a puppy? The IDs were handed out by the order of BPE's merges. Their sizes mean nothing.
- L20 hit the same wall with one-hot vectors: every word equally far from every other, so "cat" is no closer to "dog" than to "Tuesday".

What we want: one list of numbers per token, where tokens used in similar ways get similar numbers. And we want the model to **learn** those numbers, not us to choose them.

By hand: a one-hot vector times a matrix picks a row

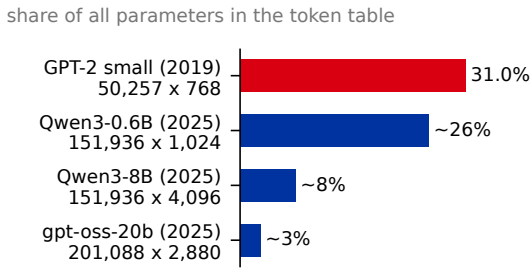
A 4-token vocabulary (the, cat, sat, on), 3 numbers per token. The table E has one row per token. "cat" is token 1, so its one-hot vector has a 1 in place 1:

$$\underbrace{\begin{pmatrix}0 & 1 & 0 & 0\end{pmatrix}}_{\text{one-hot(cat)}} \underbrace{\begin{pmatrix}0.2 & -0.1 & 0.5\\ 0.9 & 0.3 & -0.4\\ -0.6 & 0.8 & 0.1\\ 0.0 & -0.5 & 0.7\end{pmatrix}}_{E\ (4\ \text{tokens}\times 3\ \text{numbers})} = 0\cdot\text{row}_0+1\cdot\text{row}_1+0\cdot\text{row}_2+0\cdot\text{row}_3 = \begin{pmatrix}0.9 & 0.3 & -0.4\end{pmatrix}$$

- Every zero wipes out a row; the single 1 keeps row 1. So the "embedding layer" is a matrix multiplication that is really a **table lookup**: code just reads row 1.
- Training works the same way: the gradient reaches only the rows that were looked up. A token that never shows up in training never moves.

GPT-2's table

- One row per token: **50,257** rows.
- 768 numbers per row.
- $50{,}257 \times 768 = 38{,}597{,}376$ numbers: **31.0%** of GPT-2 small's 124M parameters (LLM-1's vocabulary trade-off).



Bigger models spend a smaller share on it: the table grows with the vocabulary and the width, the rest of the model grows much faster.

Qwen3 and gpt-oss: vocabulary and width from their config.json, totals from their model cards (rounded there, hence "~"), checked 4 Oct 2026.

Who fills the table?

- At the start: small random numbers. “cat” and “dog” are as unrelated as any two rows.
- Then the table trains with the rest of the model, by backpropagation, on one task: predict the next token. Nobody ever tells it what a cat is.
- You have built one already: the ch11 name inventor was **an embedding table plus an MLP**. Its table had 40 rows (38 letters, plus the start and end symbols). Same idea, now with 50,257 rows.

If two tokens keep appearing in the same places, the gradient keeps pushing their rows in the same direction. Similar use $\Rightarrow$ similar rows. Let us check whether that happened.

Which rows of GPT-2's table sit closest to " Monday"?

Closest = largest cosine similarity (the angle between two rows).

Commit to an answer.

What the table learned

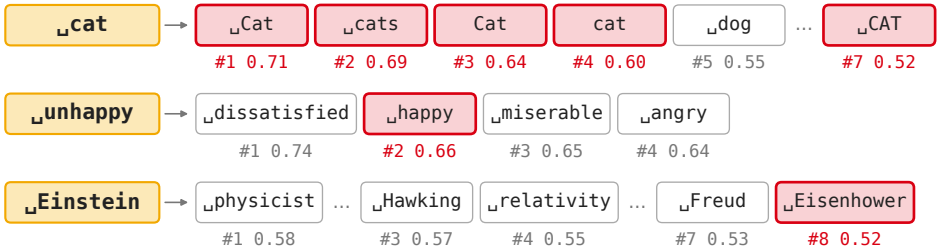
␣Monday		␣1999		␣Harry		␣Batman	
Tuesday	0.89	1998	0.89	Hermione	0.63	Superman	0.68
Thursday	0.85	1997	0.86	Voldemort	0.61	Gotham	0.62
Wednesday	0.84	1995	0.84	Ginny	0.59	superhero	0.60

␣Messi		␣lol		␣Armenia		␣vodka	
Ronaldo	0.66	haha	0.79	Armenian	0.76	whiskey	0.70
Guardiola	0.61	:)	0.64	Azerbaijan	0.72	booze	0.68
Juventus	0.60	;)	0.62	Kazakhstan	0.60	bourbon	0.66

The three nearest rows of each word, with their cosine similarity. Days next to days, years next to years, Hermione next to Harry, Ronaldo next to Messi. Spelling variants of the word itself are skipped here (next slide).

Nobody labelled a single token. All of this came from predicting the next one.

Neighbours share contexts, not meanings



- " cat": the nearest rows are the same word spelled differently. LLM-1's tokenizer gave them separate IDs; training pulled their rows back together.
- " unhappy": its second neighbour is its **opposite**. "I was ___ with the result" takes both. Opposites live in the same places.
- " Einstein": physicists, then at #8 " Eisenhower": another famous 20th-century name, with similar letters. Not physics.

A row records how a token is **used**. Usually that tracks meaning. Not always.

The glitch tokens found each other



- LLM-1's glitch token's nearest rows are **other glitch tokens**: Reddit user names and scraped-page leftovers.
- The 20 rows closest to the table's average row: these three, plus 17 single-byte tokens.
- Why: these tokens were in the tokenizer's training text and almost never in the model's. Their rows got almost no gradient (the lookup slide), so they stayed close to where they started, close to each other.

Land & Bartolo (2024), "Fishing for Magikarp", EMNLP: thousands of such under-trained tokens in today's open models, found from the weights alone.

Directions carry meaning (mostly)

Take row *b*, subtract row *a*, add row *c*, and ask which row is nearest:

query	nearest row	cosine
king − man + woman	**queen**	0.69
actor − man + woman	**actress**	0.82
cats − cat + mouse	**mice** (an irregular plural)	0.70
German − Germany + Armenia	**Armenian**	0.79
better − good + bad	**worse**	0.63
Paris − France + Italy	"Paris" (no space), Milan; Rome only 5th	0.62
walked − walk + swim	swimming, not swam	0.61

Fine print: the three input words are removed from the answers, as everyone does (Mikolov et al., 2013). Without that rule, 3 of the 5 green rows answer with one of their own inputs: king − man + woman is nearest to **king** (Nissim et al., 2020, make this point). So: a real tendency of the table, not a law.

One row per token, whatever the sentence

- "We sat on the river **bank**" and "I opened a **bank** account": both "bank"s are token 3331, so both get **the same row**.
- The table cannot see the sentence. It answers "which token is this?", never "what does it mean here?".
- Telling the two banks apart is the job of the next layers, which mix each vector with its neighbours (LLM-3).

The table gives each token a **starting point**. The vectors that come out of the last layer are different for every occurrence of a token; the ones that go in are not.

The same table, used twice

- At the very end, the model must turn its last vector back into a token: score the vector against every token, pick from the scores (LLM-5, then LLM-6).
- GPT-2 scores it against **the same 50,257 rows**: one table for reading tokens in and for writing them out. This is called **tying** the input and output embeddings.
- Small models tie, big ones usually keep two tables. Tied: Qwen3-0.6B, Gemma 4 E2B. Not tied: Qwen3-8B, gpt-oss-20b, DeepSeek-V3.

Sharing works because both jobs want the same thing: tokens used alike get vectors alike. Why the split: in a small model the table is a quarter or more of all parameters (the GPT-2 slide), so sharing it saves a lot. In a big one it is a few percent.

`tie_word_embeddings` in each model's config.json, checked 4 Oct 2026.

Where is the order?

A table lookup has no idea where the token sat.

Back to the dog

- " dog" is the 2nd token of "The dog bites the man" and the 5th of "The man bites the dog". The table hands back the same row both times.
- So after the lookup, each sentence is just **five vectors**, and they are the same five. The model has received a **set**: which vectors, but not in what order.
- "Dog bites man" is not news. "Man bites dog" is. The difference is entirely in the order.

Writing the vectors in a list on the slide gives them an order. The question is whether anything **inside the model** can tell which one was first.

Why the next layer cannot recover it

A preview of LLM-3. The next layer updates each vector like this:

1. compare it with every other vector: a **dot product** of their numbers;
2. turn those scores into weights;
3. add the vectors up with those weights.

Each step uses only **what is in** the vectors. None of them reads a slot number, and a weighted sum gives the same total whatever order you add in. Next slide: by hand. The attention lecture proves it in general: shuffle the inputs and every output comes back unchanged, just shuffled.

An RNN never had this problem: it read tokens one at a time, so order was built into the computation (L20). Attention reads all tokens at once - that is why it is fast - and the price is that order is gone.

By hand: shuffle the sentence, and nothing changes

Toy vectors with 2 numbers: dog $= (1, 0)$, bites $= (1, 1)$, man $= (0, 1)$. The next layer, simplified (LLM-3 adds learned matrices), updates "dog":

	dog bites man			**man bites dog**		
slot	vector	score	weight	vector	score	weight
0	dog $(1, 0)$	1	0.42	man $(0, 1)$	0	0.16
1	bites $(1, 1)$	1	0.42	bites $(1, 1)$	1	0.42
2	man $(0, 1)$	0	0.16	dog $(1, 0)$	1	0.42

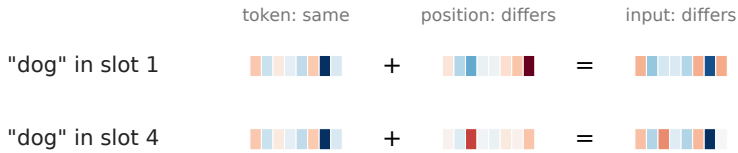
1. Score: the dot product with dog's vector. dog $\cdot$ bites $= 1 \cdot 1 + 0 \cdot 1 = 1$.
2. Weight: softmax of the scores, as in multiclass classification: e^1, e^1, e^0 divided by their sum gives $0.42, 0.42, 0.16$.
3. Output: $0.42 \cdot (1, 0) + 0.42 \cdot (1, 1) + 0.16 \cdot (0, 1) = (\mathbf{0.84},\ \mathbf{0.58})$ - in **both** sentences.

Each weight travels with its vector, not with its slot. Shuffling the sentence only shuffles the rows of this table.

The fix: write the slot into the vector

Give every **slot** its own vector too, and add it to the token's row:

$$\text{input}_i = \underbrace{E[\text{token}_i]}_{\text{which token}} + \underbrace{P[i]}_{\text{which slot}}$$



cosine of the two inputs:
0.88: close, not equal

Now "dog in slot 1" and "dog in slot 4" are different inputs, and anything that compares vectors can tell them apart. Real GPT-2 rows above: the first 8 of 768 numbers.

By hand: add positions, and dog's output changes

Same toy, plus a vector per slot: slot 0 $= (0.5, 0)$, slot 1 $= (0, 0)$, slot 2 $= (0, 0.5)$. Add them, then run the same three steps for "dog":

	dog bites man			**man bites dog**		
slot	input	score	weight	input	score	weight
0	dog $(1.5, 0)$	2.25	0.63	man $(0.5, 1)$	1.00	0.25
1	bites $(1, 1)$	1.50	0.30	bites $(1, 1)$	1.50	0.42
2	man $(0, 1.5)$	0	0.07	dog $(1, 0.5)$	1.25	0.33
	dog's output: $(\mathbf{1.25}, \mathbf{0.40})$			dog's output: $(\mathbf{0.87}, \mathbf{0.84})$		

"dog" is now a different vector in each sentence ($(1.5, 0)$ in slot 0, $(1, 0.5)$ in slot 2), so every score changes, so the weights and the output change.

Same words, different outputs: the layer can now tell who bit whom. Yet the three inputs still add up to $(2.5, 2.5)$ in both sentences. That is the trap on the next slide.

Trap: adding positions does not fix a sum

Take the cold open again, with position vectors added this time, and add up the five inputs:

$$\sum_i \big(E[\text{token}_i] + P[i]\big) \;=\; \sum_i E[\text{token}_i] \;+\; \sum_i P[i]$$

Both sentences have the same tokens and the same five slots, so both sums are the same. We measured it on GPT-2: the two totals still agree to the last bit. The toy said the same: $(2.5, 2.5)$ twice.

Positions do not change the **total**; they change **each vector**. That is enough, because the next layer compares vectors pair by pair before it adds anything up (last slide: dog's output moved from $(1.25, 0.40)$ to $(0.87, 0.84)$). A model that only summed or averaged its inputs would stay blind to order.

Option 1: learn the positions

Another table, one row per slot.

GPT-2's position table

- A second table, P: one row per slot, 768 numbers per row, trained by backpropagation exactly like the token table.
- It has **1,024** rows: 786,432 numbers, 0.63% of GPT-2. Cheap.
- Slot 1,025 has no row. GPT-2 cannot read a 1,025th token at all: the limit is built into the shape of a matrix. This limit is the model's **context length**: the most tokens it can read at once.
- BERT (2018) did the same with 512 rows. Longer documents had to be cut into pieces.

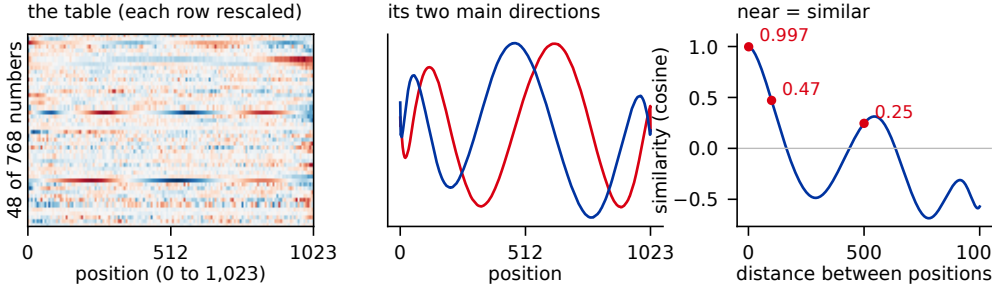
Nobody told GPT-2 that slot 7 comes between slots 6 and 8. To the training procedure they are 1,024 unrelated rows.

After training, what do those 1,024 rows look like?

Random noise? Something smooth? Something else?

Commit to an answer.

It learned waves on its own



- Right: average cosine between two slots' rows, by distance. Neighbours **0.997**, 100 apart 0.47, then it oscillates.
- Middle: PCA (ch10) on the 1,024 rows. 3 components hold 90% of the variance; the two biggest, plotted over the slots, are slow waves.

Order was learned from scratch, smooth and wave-like. Remember that for option 2.

Learned positions: the bill

- **A hard length limit.** The table has as many rows as the longest training text, and no row exists beyond it.
- **Nothing carries over.** Slot 2,000 cannot be guessed from slots 1 to 1,024: a new row needs training data at that length.
- **Absolute, not relative.** "Three slots back" has to be learned separately for slot 10, slot 500 and slot 1,000. Nothing in the setup says they are the same idea.

The waves on the last slide suggest a cheaper route: if smooth waves are what the model wants, write them down instead of learning them.

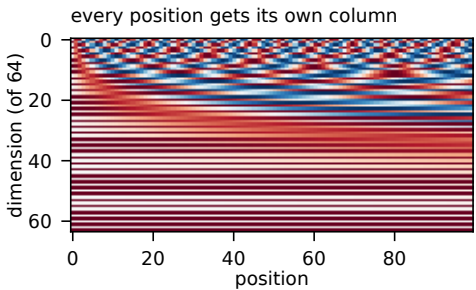
Option 2: write the positions down

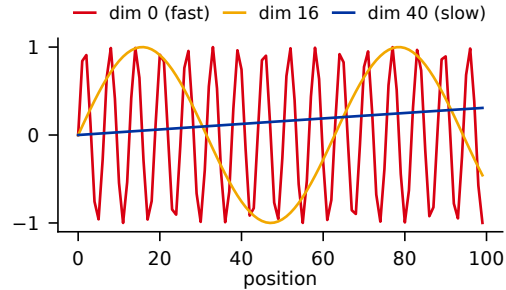
No training, and a value for every slot.

A clock with many hands

The original Transformer (Vaswani et al., 2017) fixes P with a formula. For slot *pos* and dimension pair i (of d dimensions):

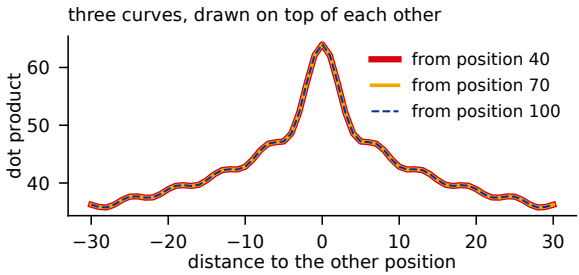
$$P_{pos,\,2i} = \sin\Big(\frac{pos}{10000^{2i/d}}\Big) \qquad P_{pos,\,2i+1} = \cos\Big(\frac{pos}{10000^{2i/d}}\Big)$$





Each pair of dimensions is a clock hand turning at its own speed. Fast hands tell neighbours apart; slow hands tell far-apart slots apart. Together they give every slot a unique reading.

Similarity depends on the gap only



The dot product of two slots' vectors, starting from slots 40, 70 and 100: the three curves are **identical** (largest difference below 10^{-9}). Only the distance matters, not where you are.

So "three slots back" looks the same at slot 10 and at slot 1,000: learn it once.

The formula has a value for every slot, so it seems to promise texts longer than any seen in training. In practice it does not deliver: trained on 512 or 1,024 tokens, such models get worse soon after the training length (Press et al., 2022). The model never learned to use the unseen readings.

Why only the gap: one identity

Take one pair of dimensions, with speed ω, at slots a and b. Its share of the dot product $P_a \cdot P_b$ is

$$\sin(\omega a)\sin(\omega b) + \cos(\omega a)\cos(\omega b) \;=\; \cos\big(\omega(a-b)\big)$$

(the cosine-of-a-difference identity). Add up all $d/2$ pairs:

$$P_a \cdot P_b \;=\; \sum_i \cos\big(\omega_i\,(a-b)\big), \qquad \omega_i = \frac{1}{10000^{2i/d}}$$

- Only $a - b$ is left: the address cancels. That is why the three curves were one.
- At gap 0 every term is $\cos 0 = 1$, so the dot product peaks at $d/2$: 64 in the plot, where $d = 128$.
- Further apart, the pairs turn at different speeds and drift out of step; their terms partly cancel. Those are the wobbles.

Hold on to $\cos(\omega(a-b))$: RoPE (option 3) is built on exactly this identity.

Every model here adds the position vector to the token vector. Why not put them side by side?

Adding mixes two kinds of information into the same 768 numbers. Surely that blurs the token?

Commit to an answer.

By hand: adding can be concatenation in disguise

A toy with 4 numbers per vector. Tokens use the first two, slots the last two:

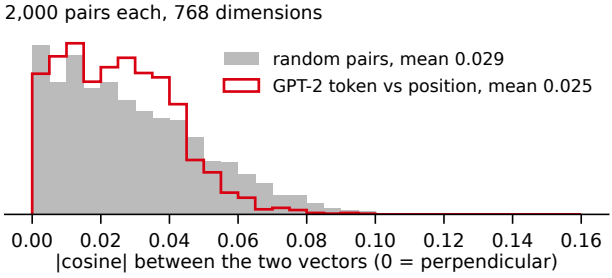
$$\text{dog} = (3, 1, 0, 0) \quad \text{cat} = (1, 3, 0, 0) \qquad \text{slot } 1 = (0, 0, 1, 0) \quad \text{slot } 4 = (0, 0, 0, 1)$$

$$\text{dog in slot } 4 = (3, 1, 0, 0) + (0, 0, 0, 1) = (3, 1, 0, 1)$$

- The first two numbers still say "dog": the dot product with dog is $3 \cdot 3 + 1 \cdot 1 = 10$, with cat only $3 \cdot 1 + 1 \cdot 3 = 6$.
- The last two numbers say "slot 4". Nothing was blurred.
- Putting them side by side, $(3, 1 \,|\, 0, 1)$, is the same thing, except that every later layer must be wider to accept the longer vector.

If tokens and slots live in **different directions**, adding them loses nothing. The question is whether a real model keeps them apart.

Measured: 768 dimensions leave room



GPT-2's token rows and position rows are **almost perpendicular**: average $|\cos|$ 0.025, even closer than random pairs (0.029).

- Add a random position to a random token, then ask which token row is nearest: the right one, **1,998 times out of 2,000**.
- The 2 misses: a stray byte and " RandomRedditor", a glitch token. The barely-trained rows again.

In high dimensions, almost every pair of directions is nearly perpendicular. Adding is cheap and keeps both messages readable.

Option 3: rotate, do not add

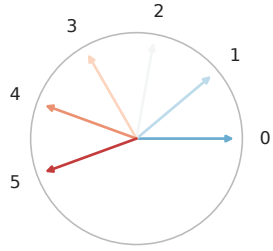
What a comparison needs is the gap, not the address.

What we actually want

- The next layer compares pairs of tokens (LLM-3). From each token's vector it makes two new ones: a **query** ("what am I looking for?") and a **key** ("what do I offer?"). The score of a pair is query · key, the dot product from the toy.
- For that score, what matters is "these two are 3 apart", not "one is in slot 41 and the other in slot 44".
- Adding a position vector bakes in the **address**. Sinusoids make the gap recoverable, but the model still has to learn to dig it out of the sum.
- Better: build the comparison so that **only the gap can affect it**.

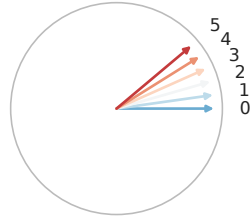
Rotary Position Embedding (RoPE; Su et al., 2021): instead of adding a position vector, **rotate** the vector by an angle proportional to its slot.

Each pair of numbers is a clock hand



a fast pair: 40 degrees per slot

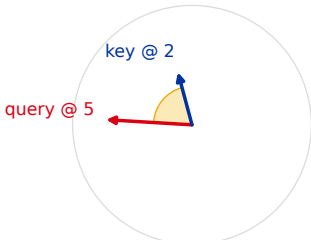
same token,
slots 0 to 5



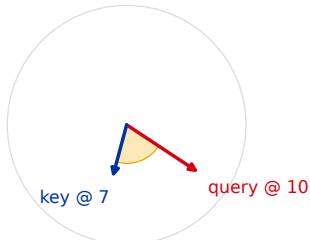
a slow pair: 8 degrees per slot

- Split the vector into pairs of numbers. Treat each pair as an arrow in 2-D.
- In slot m, turn every arrow by $m \times \theta$. Pair i has its own speed $\theta_i = 1/10000^{2i/d}$, the sinusoids' speeds: with 128 numbers per head, 64 pairs, from 1 radian per slot down to 0.000115.
- The arrow's length never changes: only its direction carries the slot.

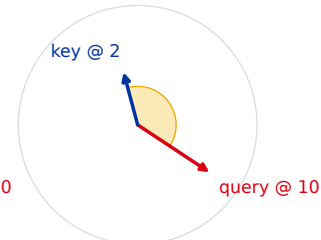
Why only the gap survives



gap 3: score 1.00



gap 3: score 1.00



gap 8: score -2.37

A dot product depends on two lengths and the angle between the arrows: $\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}|\,|\mathbf{k}| \cos(\text{angle between})$. Turn the query by $m\theta$ and the key by $n\theta$: the angle between them changes by $(m - n)\,\theta$. The lengths do not change at all. So the score depends on the content and the **gap** $m - n$ only.

By hand: same gap, same score

$\mathbf{q} = (2, 1)$, $\mathbf{k} = (1, 1)$, $\theta = 30^\circ$ per slot. Unrotated, $\mathbf{q} \cdot \mathbf{k} = 2 + 1 = 3$. Turning (x, y) by an angle φ gives $(x \cos\varphi - y \sin\varphi,\ x \sin\varphi + y \cos\varphi)$.

slots	**query turned by**	**key turned by**	**score**
q in 5, k in 2	150°: $(-2.23,\ 0.13)$	60°: $(-0.37,\ 1.37)$	$0.82 + 0.18 = \mathbf{1.00}$
q in 10, k in 7	300°: $(1.87,\ -1.23)$	210°: $(-0.37,\ -1.37)$	$-0.68 + 1.68 = \mathbf{1.00}$
q in 10, k in 2	300°: $(1.87,\ -1.23)$	60°: $(-0.37,\ 1.37)$	-2.37

Gap 3 gives 1.00 at slots 5/2 and at 10/7. Gap 8 gives something else. Check one row yourself: $\cos 150^\circ = -0.866$, $\sin 150^\circ = 0.5$, so the query becomes $(2 \cdot (-0.866) - 1 \cdot 0.5,\ 2 \cdot 0.5 + 1 \cdot (-0.866)) = (-2.23,\ 0.13)$.

The model never sees the slot numbers 5, 2, 10 or 7. It only ever feels the gap.

Where the rotation happens

- Not at the input. The token vector goes in with **no** position added.
- The rotation is applied inside every attention layer, to the query and the key of each token, right before their dot product.
- So position enters each layer's comparison directly, instead of being added once at the bottom and carried through.

On the map, the Position box stays where it is: it is still "tell each vector where it sits". With RoPE it does so inside the attention box, at every layer.

Who uses it

- RoFormer (2021) reported modest gains. Adoption was anything but modest: GPT-J and GPT-NeoX, PaLM, then LLaMA (2023), and after LLaMA nearly every open model.
- Every 2024-2026 model in the table at the end of today uses RoPE in some form.
- Two reasons it won: the score depends only on the gap, and it adds no parameters at all - a rotation has nothing to learn.

One thing RoPE does not fix by itself: texts longer than the training ones. That is the last section.

Three options, side by side

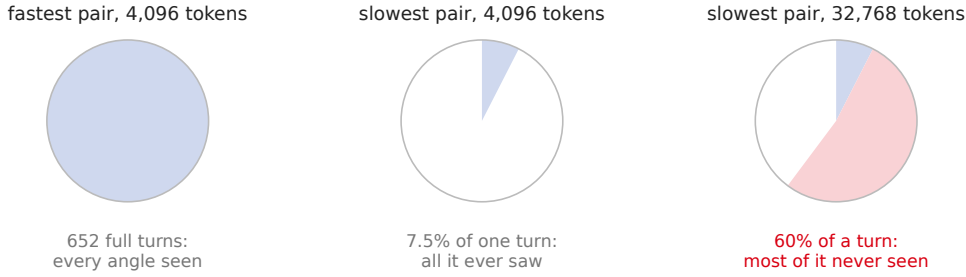
	learned table (GPT-2)	**sinusoids** (2017)	**RoPE** (2021)
what it does	adds a trained row per slot	adds a fixed wave pattern	rotates query and key inside attention
parameters	one row per slot (786,432 numbers in GPT-2)	none	none
"3 slots apart"	learned separately at every slot	the same pattern everywhere; the model must dig it out	the score sees only the gap
past the training length	impossible: no row exists	allowed, but quality drops	allowed, but quality drops; can be stretched (next section)

Each option fixes something in the column before it: sinusoids drop the trained rows and make the gap the same everywhere; RoPE makes the gap the **only** thing a comparison can see. None of them, by itself, makes a model good at lengths it never trained on.

Longer and longer

GPT-2 stopped at 1,024 tokens. Models now read hundreds of thousands.

Why RoPE breaks past its training length



A model trained on 4,096 tokens, RoPE base 10,000, 128 numbers per head (Llama 2's settings). The fastest hand spins about 650 times during one training text: it has seen every angle. The slowest turns 0.000115 radians per slot: $4{,}096 \times 0.000115 = 0.47$ radians, only **7.5%** of a circle. Feed it 32,768 tokens and that hand reaches 60%: angles the model never saw, so the scores there are a guess.

Press et al. (2022) measured it: RoPE models hold up for 100-200 tokens past their training length, then get worse. Sinusoids fail sooner.

Stretch the clock

- **Position interpolation** (Chen et al., 2023): divide every slot number by 8. Slot 32,000 is read as 4,000, an angle the model knows. But neighbours now differ by only 1/8 of a step: harder to tell apart.
- **YaRN** (Yet another RoPE extensioN; Peng et al., 2024): stretch only the **slow** hands, the ones that never completed a turn. Leave the fast hands alone; they have seen every angle anyway.
- Then a short extra training run on long documents, so the model gets used to it.

In 2025 configs: Qwen3-8B, native length 32,768 tokens, reaches 131,072 with YaRN. gpt-oss-20b was stretched from 4,096 to 131,072.

Qwen3-8B model card; gpt-oss-20b config.json (`rope_scaling`), checked 4 Oct 2026.

Two other answers: a distance penalty, or no position at all

- **Attention with Linear Biases** (ALiBi; Press et al., 2022): no position vectors at all. Each comparison score just gets a penalty that grows with the distance, so near tokens win unless a far one is much more relevant. BLOOM (2022) used it.
- **No Positional Encoding** (NoPE): nothing. A model that may only look back at earlier tokens (decoder models, LLM-5) still works out positions on its own (Haviv et al., 2022): token 3 sees 3 tokens, token 300 sees 300, and that count gives its position away.
- Llama 4 (2025) mixes RoPE and NoPE: some layers rotate, some have no position at all. Meta reports up to 10 million tokens of context for Llama 4 Scout.

Position is still an open design question. The 2017 answer did not last, and the 2021 answer keeps being patched.

What models use, 2017-2026

model	position	context (tokens)
Transformer (2017)	sinusoids	-
BERT (2018)	learned table	512
GPT-2 (2019)	learned table	1,024
T5 (2019)	a learned bias per distance	512 in training
BLOOM (2022)	ALiBi	2,048
DeepSeek-V3 (2024)	RoPE + YaRN (from 4,096)	163,840
Qwen3-8B (2025)	RoPE, base 1,000,000	32,768; 131,072 with YaRN
gpt-oss-20b (2025)	RoPE + YaRN (from 4,096)	131,072
Llama 4 Scout (2025)	RoPE layers + NoPE layers	10,000,000
Gemma 4 E2B (2026)	RoPE, two bases: 10,000 and 1,000,000	131,072

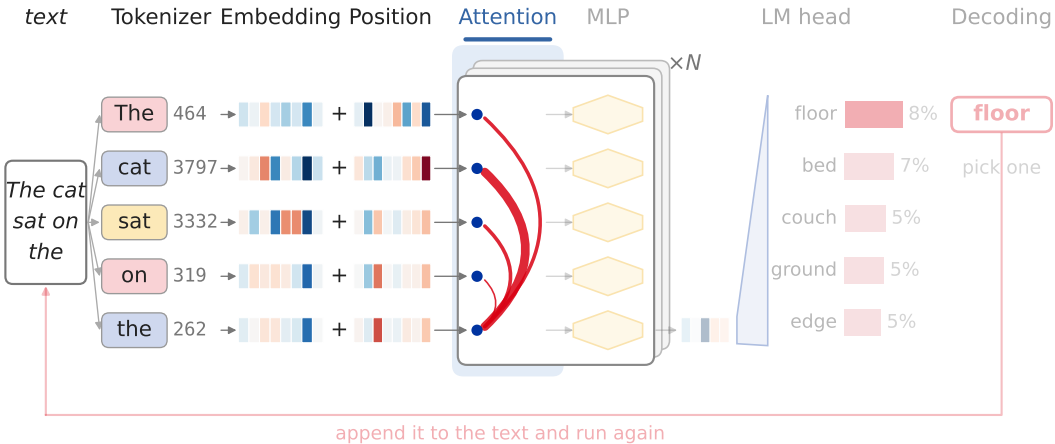
Gemma's two bases: layers that only look at nearby tokens use the faster clock (10,000), layers that see the whole text the slower one.

config.json and model cards on Hugging Face, Meta's Llama 4 announcement; checked 4 Oct 2026.

Recap

- An ID picks a **row of a learned table** (one-hot times a matrix). The rows learn how tokens are used: days near days, opposites near each other, glitch tokens in a pile.
- A lookup ignores order: both dog sentences arrive as the same five vectors.
- So each vector also gets its **slot**: a learned row (GPT-2, hard limit 1,024), a formula (sinusoids), or a rotation inside every attention layer (RoPE: only the gap counts).
- Adding works because in 768 dimensions token and position directions barely overlap.
- Longer texts: stretch the slow clock hands (YaRN), or penalise distance (ALiBi), or drop position in some layers (NoPE).

Next on the map



Next (LLM-3): the vectors start talking to each other.

Sources & further reading

- Vaswani et al. (2017), "Attention Is All You Need", NeurIPS (sinusoids).
- Su et al. (2021), "RoFormer: Enhanced Transformer with Rotary Position Embedding", arXiv; Neurocomputing (2024).
- Press, Smith & Lewis (2022), "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation", ICLR.
- Haviv et al. (2022), "Transformer Language Models without Positional Encodings Still Learn Positional Information", Findings of EMNLP.
- Chen et al. (2023), "Extending Context Window of Large Language Models via Positional Interpolation", arXiv; Peng et al. (2024), "YaRN", ICLR.
- Mikolov, Yih & Zweig (2013), "Linguistic Regularities in Continuous Space Word Representations", NAACL; Nissim, van Noord & van der Goot (2020), "Fair is Better than Sensational", Computational Linguistics.
- Land & Bartolo (2024), "Fishing for Magikarp", EMNLP. Meta (April 2025), "The Llama 4 herd".

Every GPT-2 number: `py_src/embeddings_position_figs.py` (GPT-2's weights read with safetensors, IDs from tiktoken), raw results in `results/embeddings_position_figs.json`.